

SQL Window Functions — Results

All queries executed against an in-memory SQLite database using the exact schema and data provided.

Schema & Data

```
CREATE TABLE test_data (  
    new_id INT,  
    new_cat VARCHAR(255)  
);  
  
INSERT INTO test_data (new_id, new_cat)  
VALUES  
    (100, 'Agni'), (200, 'Agni'),  
    (500, 'Dharti'), (700, 'Dharti'),  
    (200, 'Vayu'), (300, 'Vayu'),  
    (500, 'Vayu');
```

1. Aggregate Window Functions

```
SELECT new_id, new_cat,  
SUM(new_id) OVER( PARTITION BY new_cat ORDER BY new_id ) AS "Total",  
AVG(new_id) OVER( PARTITION BY new_cat ORDER BY new_id ) AS "Average",  
COUNT(new_id) OVER( PARTITION BY new_cat ORDER BY new_id ) AS "Count",  
MIN(new_id) OVER( PARTITION BY new_cat ORDER BY new_id ) AS "Min",  
MAX(new_id) OVER( PARTITION BY new_cat ORDER BY new_id ) AS "Max"  
FROM test_data;
```

new_id	new_cat	Total	Average	Count	Min	Max
100	Agni	100	100	1	100	100
200	Agni	300	150	2	100	200
500	Dharti	500	500	1	500	500
700	Dharti	1200	600	2	500	700
200	Vayu	200	200	1	200	200
300	Vayu	500	250	2	200	300
500	Vayu	1000	333.333	3	200	500

These are running (cumulative) aggregates within each partition, ordered by new_id — not full-partition totals. Each row's Total/Average/Count reflect everything from the start of that category up through the current row, which is why Agni's second row shows Total=300 rather than a fixed group total.

2. Ranking Window Functions

```
SELECT new_id,  
ROW_NUMBER() OVER(ORDER BY new_id) AS "ROW_NUMBER",  
RANK() OVER(ORDER BY new_id) AS "RANK",  
DENSE_RANK() OVER(ORDER BY new_id) AS "DENSE_RANK",  
PERCENT_RANK() OVER(ORDER BY new_id) AS "PERCENT_RANK"  
FROM test_data;
```

new_id	ROW_NUMBER	RANK	DENSE_RANK	PERCENT_RANK
100	1	1	1	0
200	2	2	2	0.167
200	3	2	2	0.167
300	4	4	3	0.5
500	5	5	4	0.667
500	6	5	4	0.667
700	7	7	5	1

ROW_NUMBER breaks ties arbitrarily (1,2,3...), RANK leaves gaps after ties (2,2,4), and DENSE_RANK does not (2,2,3) — visible directly in the two rows where new_id=200 and the two rows where new_id=500.

3. Analytic Window Functions

```
SELECT new_id,  
FIRST_VALUE(new_id) OVER( ORDER BY new_id) AS "FIRST_VALUE",  
LAST_VALUE(new_id) OVER( ORDER BY new_id) AS "LAST_VALUE",  
LEAD(new_id) OVER( ORDER BY new_id) AS "LEAD",  
LAG(new_id) OVER( ORDER BY new_id) AS "LAG"  
FROM test_data;
```

new_id	FIRST_VALUE	LAST_VALUE	LEAD	LAG
100	100	100	200	NULL
200	100	200	200	100
200	100	200	300	200
300	100	300	500	200
500	100	500	500	300
500	100	500	700	500
700	100	700	NULL	500

Note on LAST_VALUE: it does not return the true last row's value (700) for every row, the way it's often expected to. Because ORDER BY alone implies a default frame of RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW, LAST_VALUE actually returns the *current* row's value at each step. To get the real last value across the whole result set, the frame needs to be stated explicitly: LAST_VALUE(new_id) OVER (ORDER BY new_id ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) — which would return 700 for every row.